

taskIt

Project Report

Table Of Contents

Introduction	3
Problem Statement	5
Functional Requirements	6
Non-Functional Requirements	9
What Technologies Are Used	11
How It Works	12
Some Flow Diagrams	15
Use Case Diagram	17
Workflow Diagram	17
Class / Component Diagram	18
ER Diagram	19
Design Decision And Rationale	21
Implementation Details	22
Screenshots	25
Testing Approach	31
Challenges Faced	33
Learning And Key Takeaways	34
Future Enhancements	35
References	37
About	38

Introduction

“taskIt” is a simple program designed to help solve a person’s task management problems that one might face in their day to day life.

It was created by me (`real-xp` | `xp` | `26BCE10996`) for the university’s course site’s project requirement for September 2026.

It is simply designed to be efficient, simple, and most importantly, a useful productivity tool for a person who might struggle to manage his/her own time. This little program fixes that problem.

It is purely written in `Python` and uses `SQLite3` for database management.

It is a CLI [`Command Line Interface`] tool that purely works on in-built modules and custom created modular and future-proof designed modules, making it really snappy and quick.

Each and every aspect of the program has been handled with care and carefully thought out so it works just as one intends it to.

Due to how the program has been thought out and designed, anyone can read it’s code and understand it easily due to plentiful documentation and carefully designed and named functions / method and variables / attributes.

About the usage of AI -

AI was not used at all during the planning, production, or creation of this project. It was neither used for asking any kind of documentation help nor was it used for “vibe-coding” or copy-pasting of code.

Every single aspect of this program was handwritten down to each line, variable name, comment, and even implementation.

Problem And Solution

- Problem Statement
- Functional Requirements
- Non Functional Requirements

Problem Statement

In a single line or a single phrase, if a problem statement was to be described, it would be -

“Task Management without compromising speed and space”

These days, people tend to use very bulky apps and websites to manage tasks and store their data on cloud servers which might become a privacy issue. Due to this, smaller tools that can easily and efficiently store data without compromising privacy and network needs is a much-needed requirement.

`taskIt` solves that problem. It completely works offline, so no need for a bulky electron-built browser application that requires a lot of memory and CPU usage overhead.

On top of that, it is purely `CLI` based, making it really snappy and quick, not requiring any `OS` dependent UI libraries.

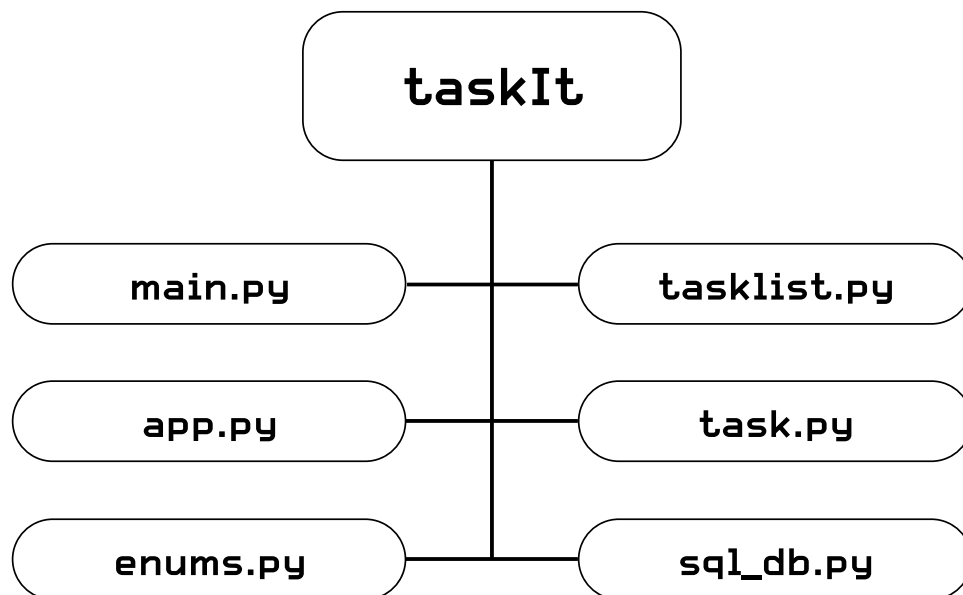
The best part about the app is its compatibility with `SQLite3` Database files. This ensures that your data stays with you at any time.

In short, the app aims to solve the issue of “a person facing time management issues and wanting to avoid bulky, subscription-filled / ad-filled, slow apps and websites for time and task management”

Functional Requirements

This app works on a modular system of multiple modules, listed below -

- **main.py** - which manages the entire brain of the project including the main loop.
- **app.py** - the brain of the entire operation. This module contains the **app** class that has a lot of methods that manage how tasks are made and how task lists are created. This basically does all the CRUD operations locally.
- **tasklist.py** - which manages the entire tasklist structure of the project by the usage of a **tasklist** class.
- **task.py** - which manages the entire task structure of the project by the usage of a **task** class.
- **sql_db.py** - which manages the database and saves / loads data. It, like other modules, uses a **database** class which is modular and can have more methods added later.



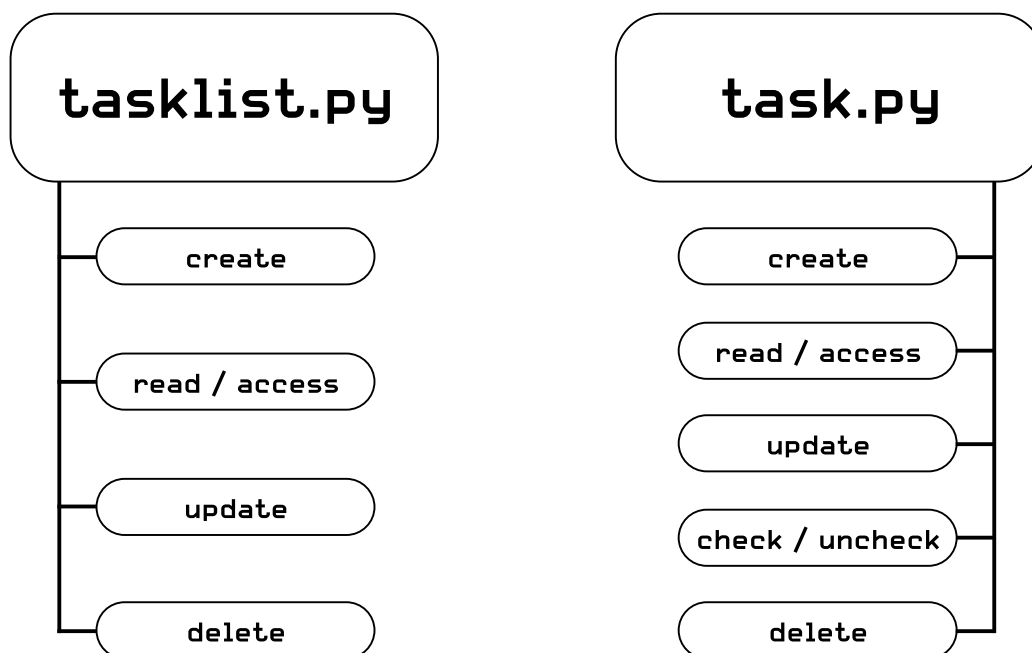
- **enums.py** - which basically stores all content / literal values like “help” statements and enumerator values in it.

In terms of the IO (Input / Output) structure, it is really simple.

- The program is in an infinite loop until broken, so inputs can be given constantly.
- An input is asked every iteration of the main loop, where user can perform several

actions depending on which menu the user is in.

- If the user is in the “main menu” or the “home page”, the user can do CRUD operations on “task lists”.
- These “task lists” are a data type that store the actual “task” inside them. These are made for more customization by a user.
- Operations you can perform on “task lists” are -
 - **create** - which allows you to create a tasklist.
 - **access / view** - which allows you to view a tasklist and its tasks, or view all task lists.
 - **update** - which allows you to update a tasklist (particularly it’s name).
 - **delete** - which allows you to delete a tasklist.
- If the user is in a “task list”, the user can do CRUD operations on tasks.
- Operations you can perform on “tasks” are -
 - **create** - which allows you to create a task. This includes the name of the task, its priority, whether it is completed or not, and some tags for more organization.
 - **access / view** - which allows you to view a task in the particular tasklist.
 - **update** - which allows you to update a task.
 - **check / uncheck** - which allows you to check or uncheck a task.
 - **delete** - which allows you to delete a task.

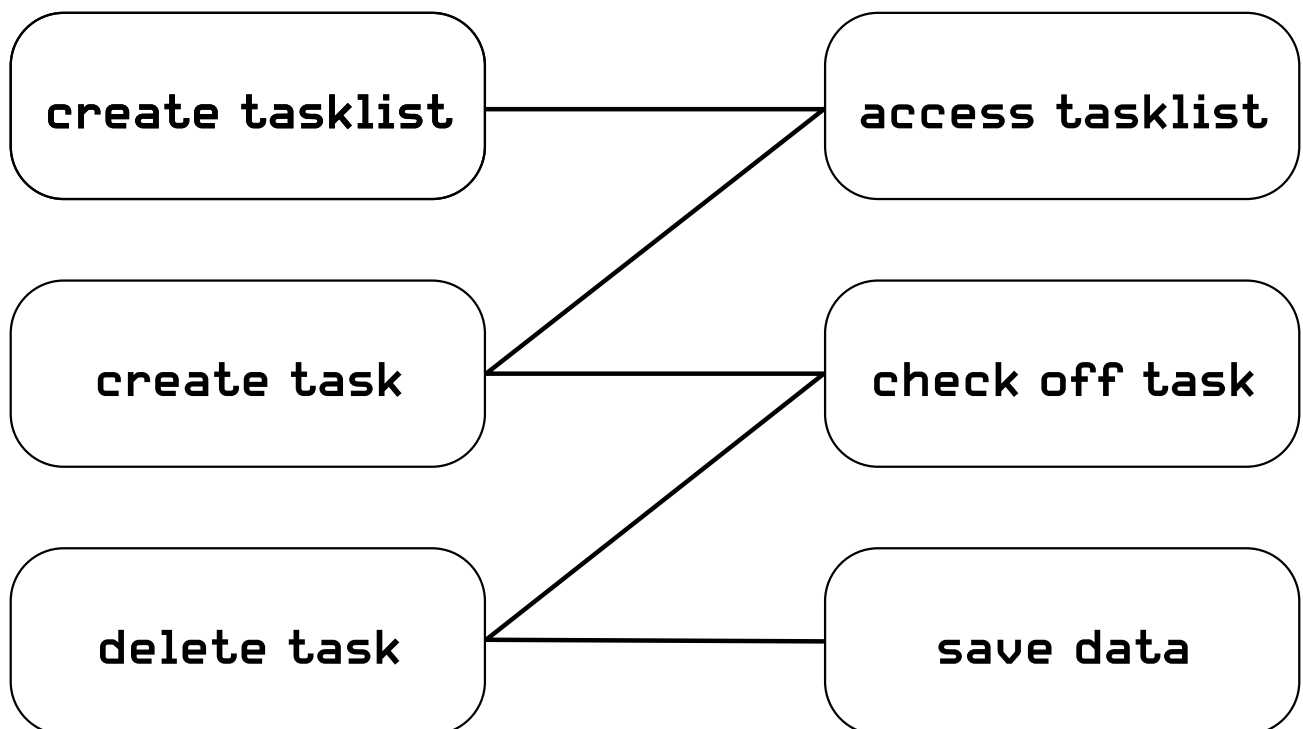


- Other than these, the user can also type `home` to go back to the main menu anytime.
- The user can also type `save` to save the data into a SQLite3 database file, which is automatically loaded on the next boot of the program.
- The user can also type `help` to see how the program works exactly.
- The user can also type `exit` / `quit` to exit the program.

Every single input the user puts in is designed to give an appropriate feedback / response / output to the end user.

In general, the user interacts with the system in the following way -

- The user first creates a tasklist to store all their tasks in.
- Then the user accesses the tasklist, and creates tasks.
- When a task is completed in the tasklist, the user can tick them off for the time being, or delete them entirely.
- When they are done with their task and tasklist operations, they can save the database and come back anytime to the same data.



Non-Functional Requirements

In terms of Non-Functional requirements prescribed in the document beforehand, this program handles (or expects) to handle the following -

- **Performance**

The program was designed to be performance first. Due to this, it works really quickly and efficiently.

- **Usability**

It is easy to use for the end user due to its linear structure of commands and modular input system across menus.

- **Reliability**

The program can be considered reliable due to its ability to write to a reliable database and retrieve from it, while also being reliable in the sense of code stability.

- **Scalability**

The program is scalable as it was designed to handle moderate amount of data.

- **Maintainability**

Due to its modular nature and self-documenting and short-simple structure, it is really easy to maintain and expand on.

- **Resource Efficiency**

It was designed to be as resource efficient as it could be.

System Architecture

- What Technologies Are Used
- How It Works
- Diagrammatic Representation

What Technologies Are Used

`taskIt` uses a mix of technologies in order to work.

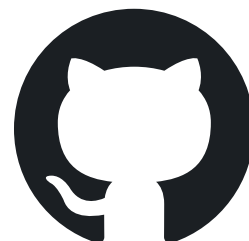
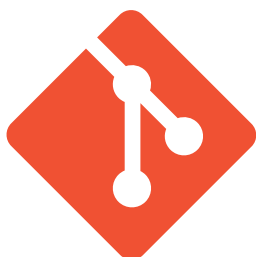
In terms of programming language, the chosen language was `Python` as not only was it easy to work in, but also was the main topic of the course this project is assigned to.

In terms of data storage, it uses a combination of `dictionaries`, `lists` and also uses `SQLite3` relational database system to store task and tasklist data. This makes it easy to scale and expand for larger data sets.

It also uses a combination of in-built modules and libraries like `date time`, `enum`, `sqlite3`, `subprocess`, `platform` and user-defined modules described before.

Other than all this, the program was written in `Visual Studio Code` with all auto-completion and `Intellisense` turned off.

`Git` was used for version control and `GitHub` was used for management of remote repository.



How It Works

First, the `main.py` file is run as the starting point of the project. This also contains the main loop of the program that is used for repeated inputs from user.

Then a user input is taken in and passed into the `app.py` module to parse each and every argument given in the statement. Example - `create;newlist` separates the parent command, `create` from the argument separated by using “;”.

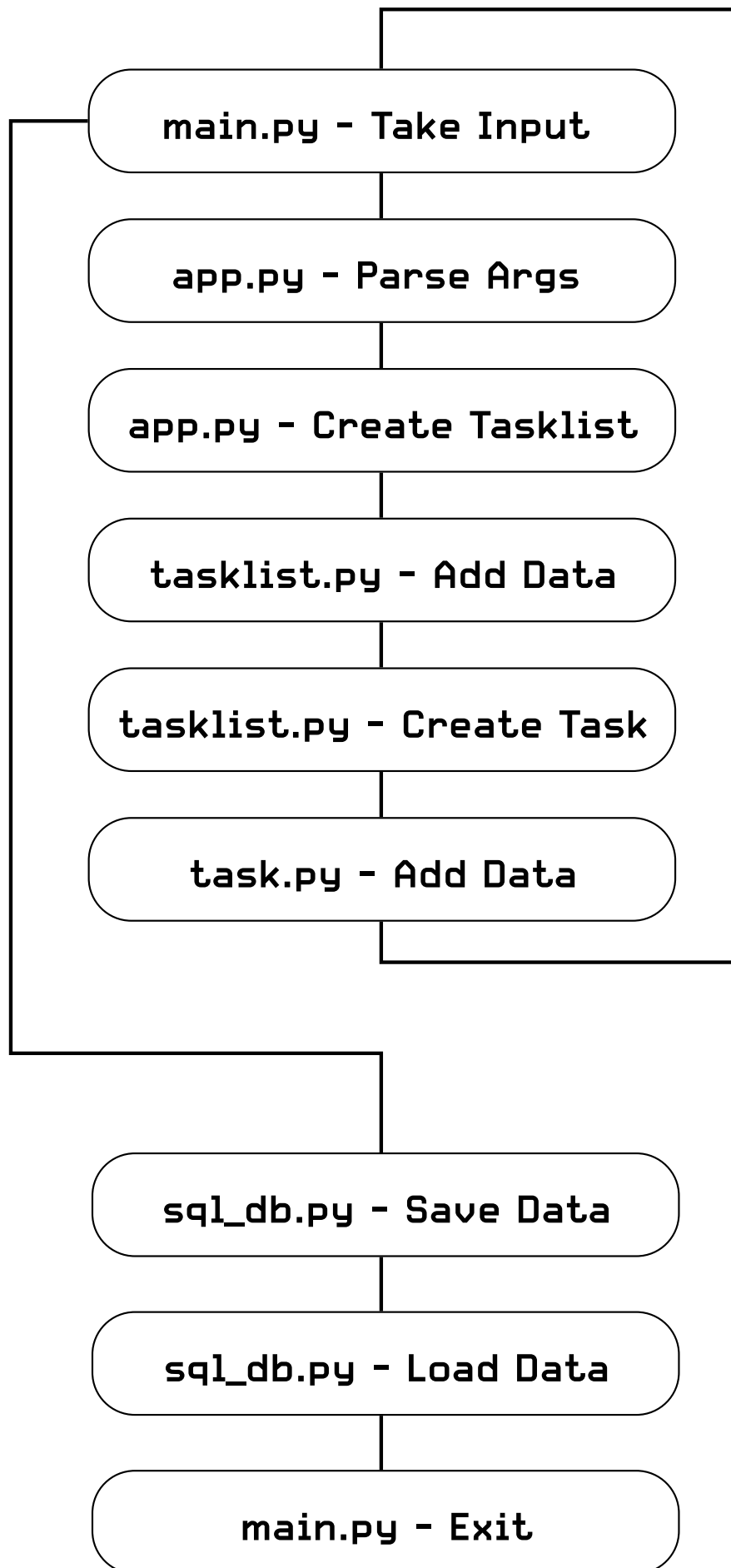
Depending on the command and its arguments, and the menu you are currently in, you can operate a few CRUD operations and other operations.

Creating a `tasklist` using the `create;newlist` command invokes a method inside the `app class` of `app.py` module, which then makes a new tasklist using the structure described in `tasklist.py` module, which also handles some other methods and things about `tasklists`.

Accessing a `tasklist` grants you the ability to work with `tasks` which you can operate CRUD operations on. Example is of creating a `tasks` using the `create` command, where it will ask you about the `name`, `if checked`, `priority`, `tags`. These tasks are then stored using the structure defined in the `task.py` module, which also manages things on tasks itself.

Let us say you make a minor mistake. The program will give an error and handle it appropriately. Every error, or a literal string statement is defined in `enums.py` module, which uses the `enum` module that is in-built.

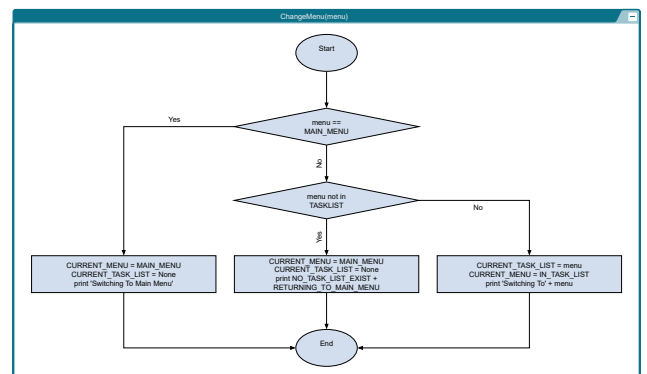
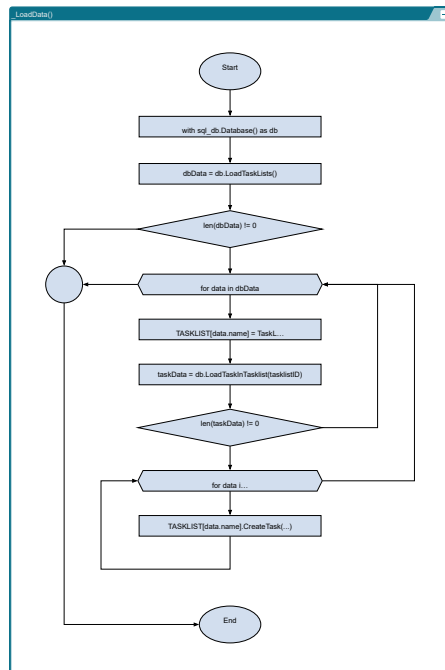
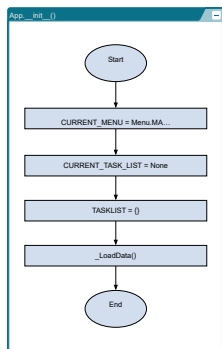
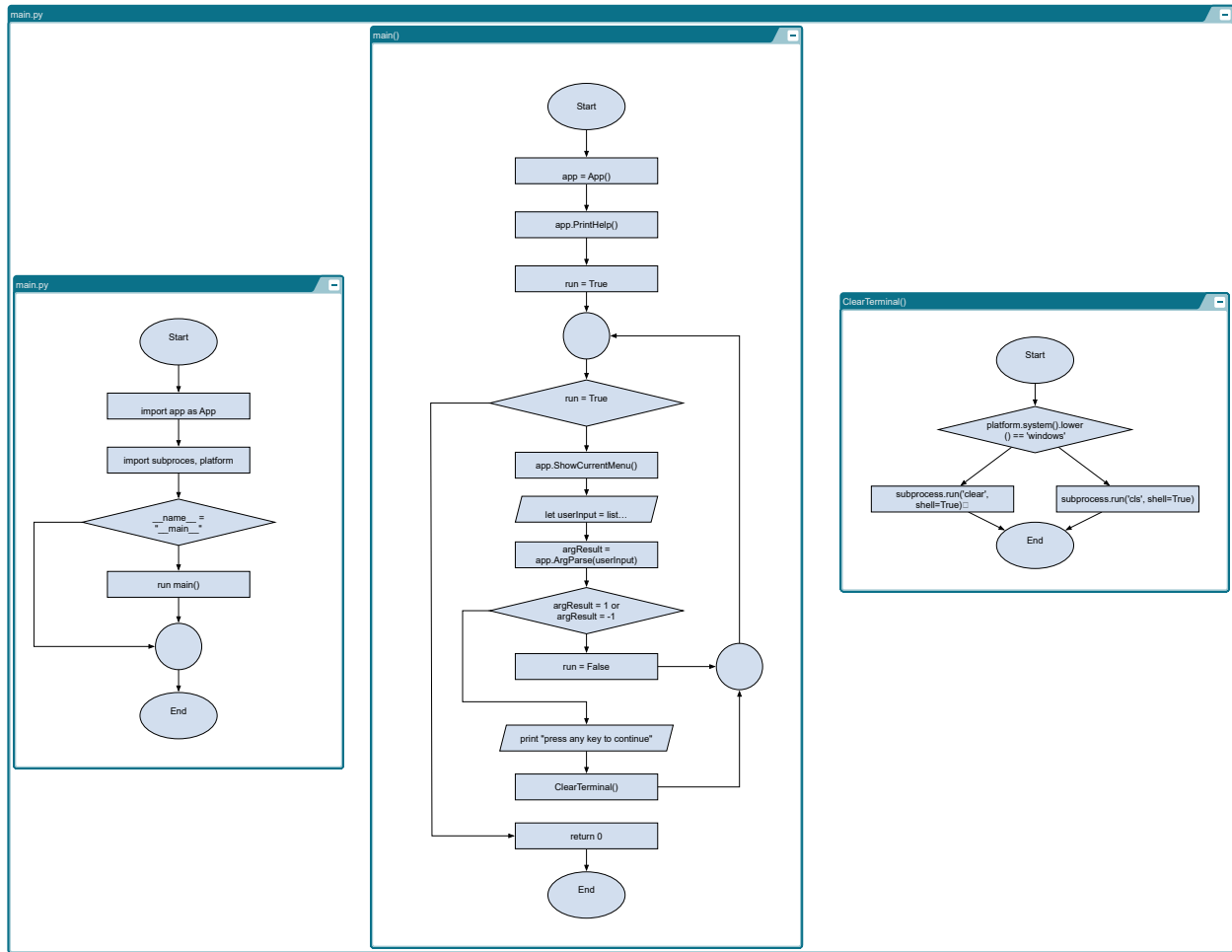
To finally save the database, one can type in the `save` command and it will call the database's `save` method in the `sql_db.py` module, which uses `SQLite3` in-built module to manage database.

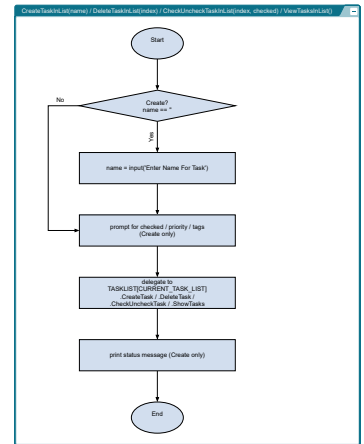
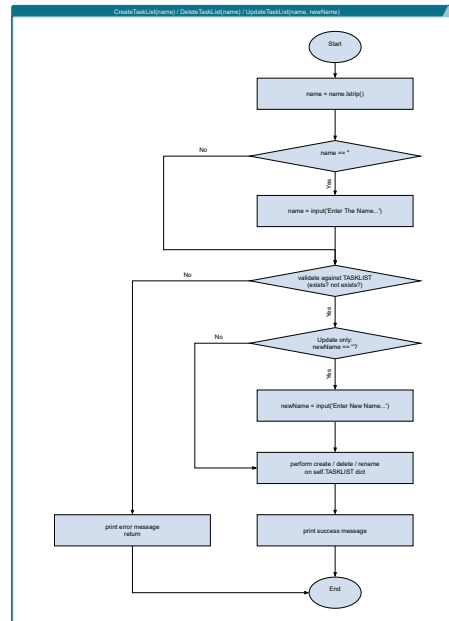
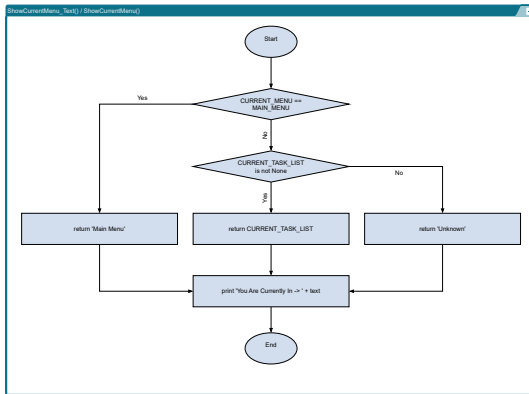


Design Diagrams

- Some Flow Diagrams
- Use Case Diagram
- Workflow Diagram
- Class / Component Diagram
- ER Diagram

Some Flow Diagrams





Please note that these are not all the flow diagrams for each and every module as every single module is really complex and would take days if not weeks to finish.

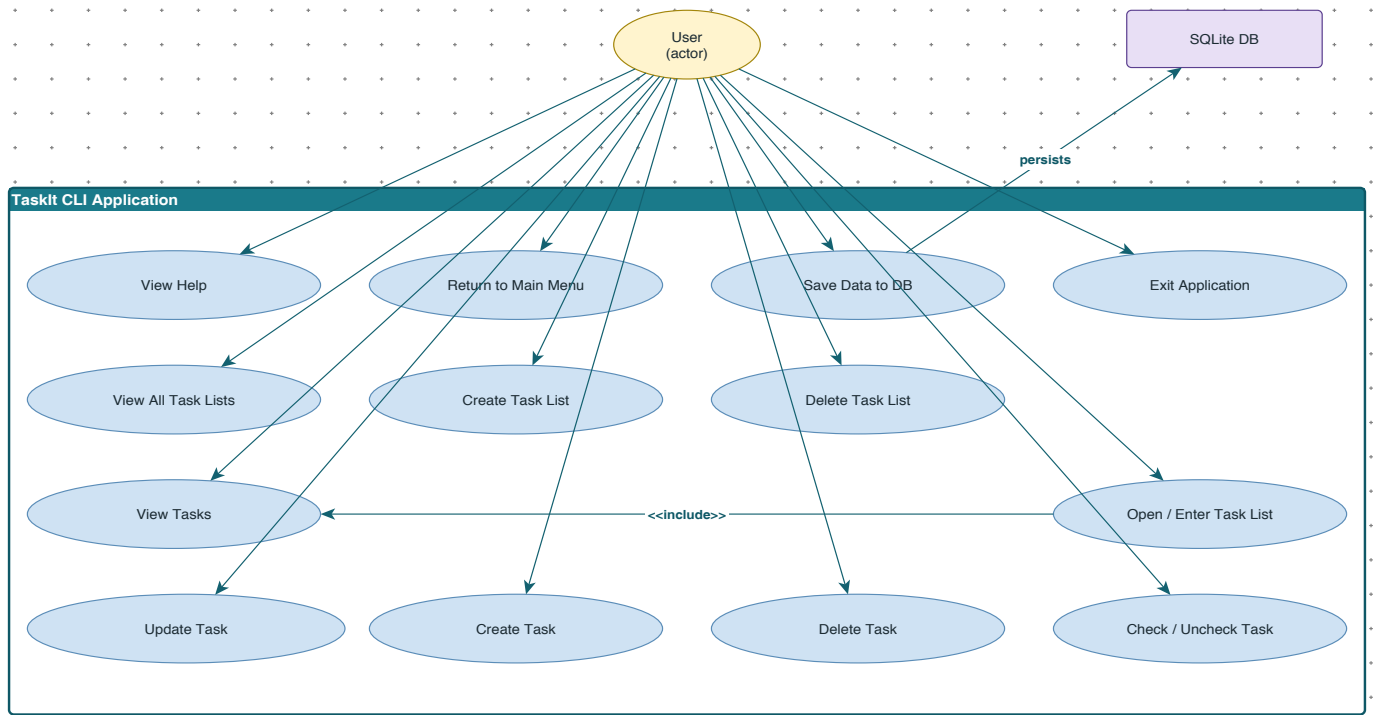
They are really complex to compute and especially draw digitally.

yEd was used for computing these graphs.

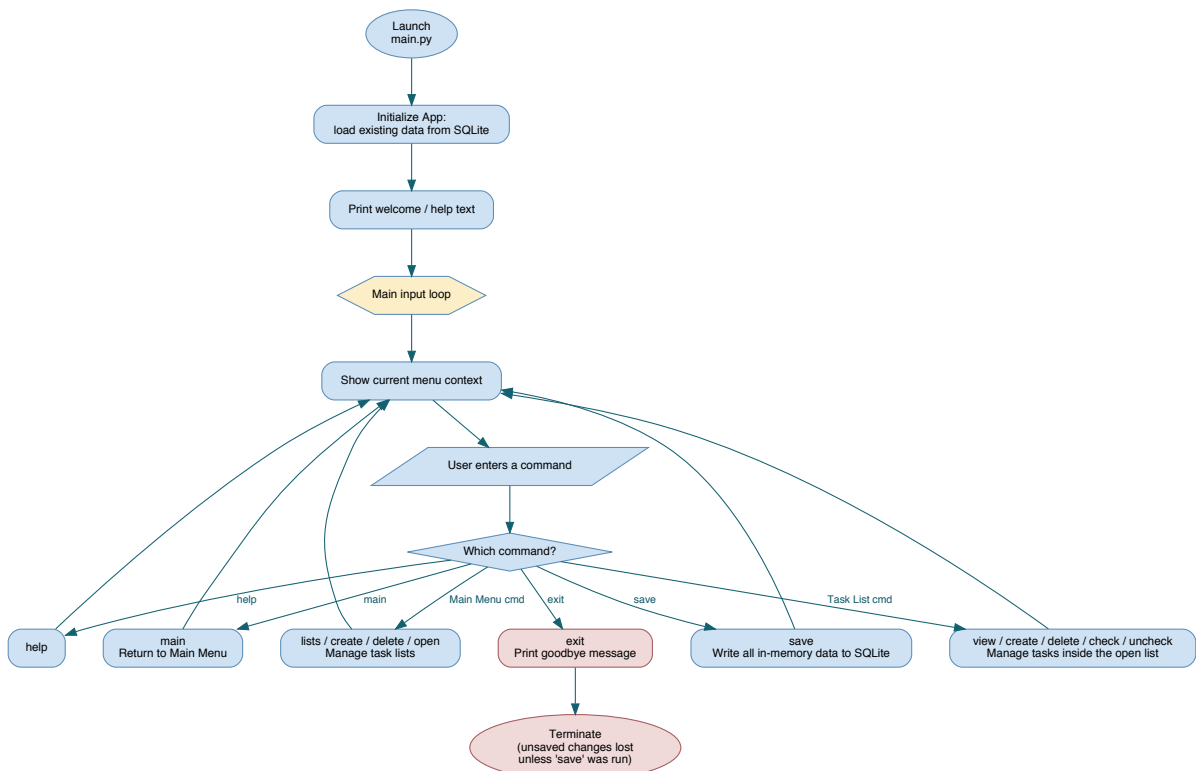
Both the live and the desktop version was used in this case.

In some cases, diagrams were hand drawn on Adobe Illustrator.

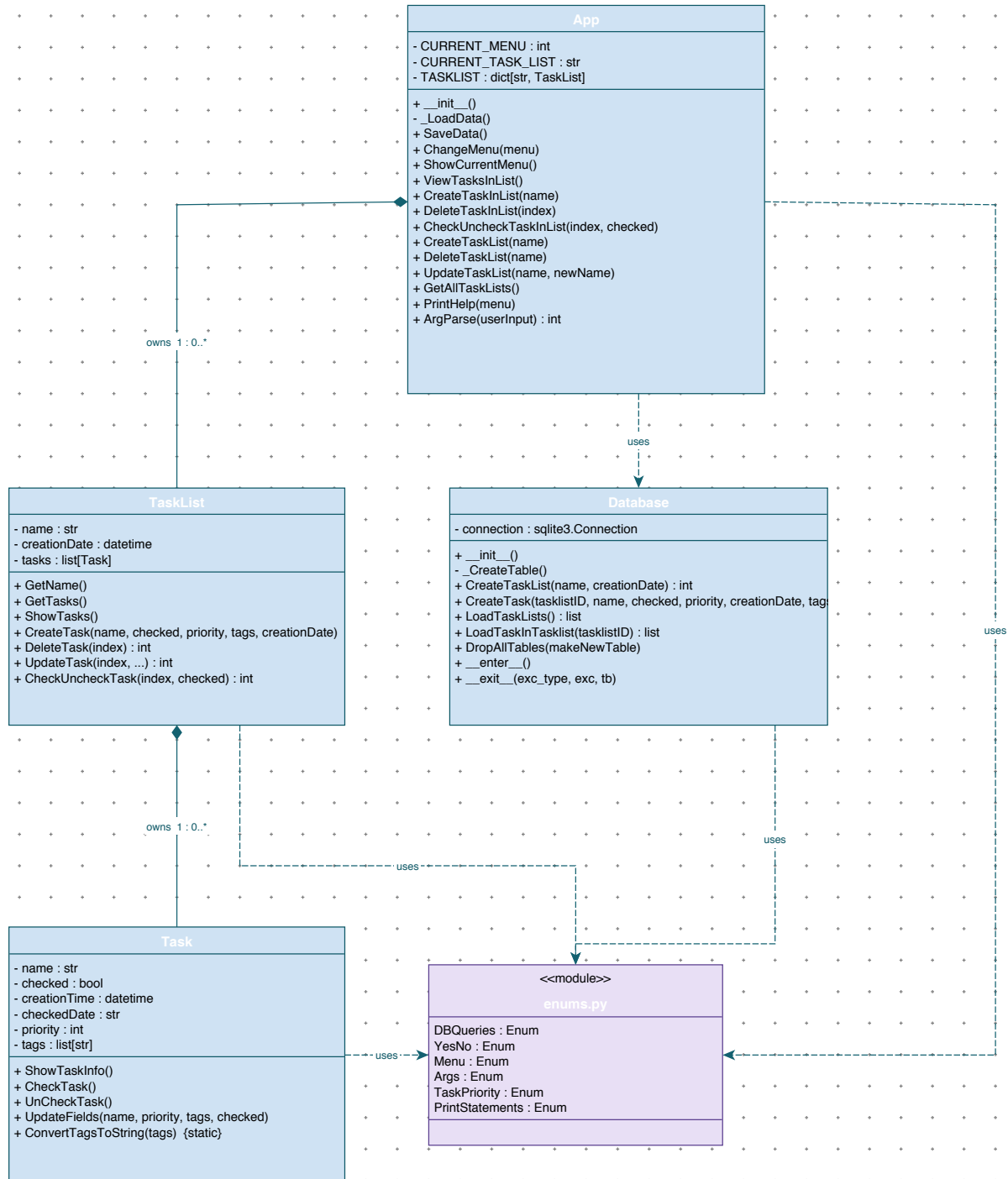
Use Case Diagram



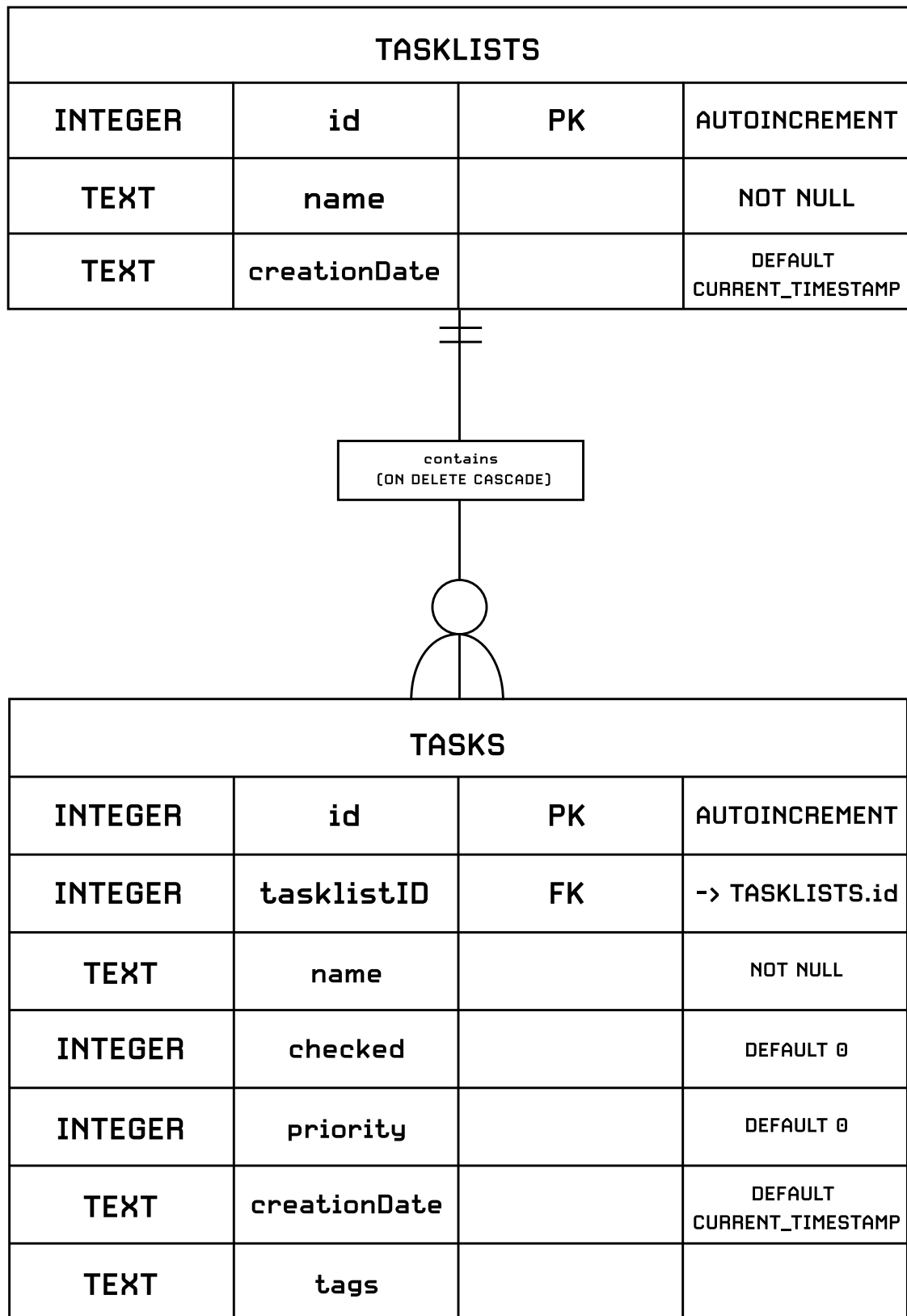
Workflow Diagram



Class / Component Diagram



ER Diagram



Design And Implementation

- Design Decision And Rationale
- Implementation Details

Design Decision And Rationale

A lot of design decisions were made before and during the creation of this project.

`CLI (Command Line Interface)` was chosen as the method of interaction as it was quick, fast, did not require any UI library to be installed, and was quick, fast and efficient.

`Python` was chosen as the main language of programming as it is easy to work with and is the main topic of the course.

`SQLite3` was chosen for database management instead of some other technology as `SQLite3` is fast, efficient, easy to implement, relational and built-in to Python.

`subprocess` and `platform` modules were chosen to perform a particular task for the CLI, and to make some parts work on all Operating Systems.

`enum` module was chosen as Enumerators are easy to work with and help in direct organization of data.

Different modules were created and each with different classes, as the program was made to be extremely modular and scalable with new features, maintainable and also easy to understand.

In general, the program was made with a single motive in mind - utility.

Implementation Details

As described before, each and every module does some kind of task that is important for the entire program to work.

- **main.py**
 - This is the first file that is ran in the entire project.
 - It manages the main loop of the program.
 - It sends the inputs directly to the `ArgParse` function in the `app.py` file.
 - The inputs are mapped to a list with the first element representing the command and the ones after it representing the arguments. Each of these is to be separated by a ;
 - In case of an error or an exit command, the loop breaks and the program is exited.
- **app.py**
 - This is the brain of the entire project.
 - It manages everything from loading and saving data into the `SQLite3 Database` to managing user inputs and errors.
 - In case of an error or an exit command, the loop breaks and the program is exited.
- **tasklist.py**
 - This is the structure for the `tasklists` object.
 - This also manages data for `tasklists` and also the `tasks` inside that `tasklist`.
- **task.py**
 - This is the structure for the `tasks` object.
 - This manages data for `tasks`.
- **sql_db.py**
 - This is the second brain of the entire data structure.

- This manages the data storage of the entire program using `SQLite3`.
- It can load, save, create, read, and delete data into a `SQLite3 Database`.

- `enums.py`

- This is the smallest yet an important module of the program.
- It basically stores the entire program's string literals and enumerations.

All these modules work together to create a seamless experience of user experience and program logic.

Screenshots And Results

Screenshots

In main menu

```
PowerShell x + v
-----
TaskIt
-----

Welcome to TaskIt! This app allows you to manage tasks and task lists.
You are currently in the MAIN MENU.

To see all the lists, when prompted, type - "view" or "lists"
To create a list, when prompted, type - "create" or "create;<name>" without <>
To access a list, when prompted, type - "open;<name>" without <>
To update a list, when prompted, type - "update;<name>;<new name>" or "update" without <>
To delete a list, when prompted, type - "delete" or "delete;<name>" without <>

To save the database into a '.db' file, type "save" when prompted.
To go back to the main menu, type "home" when prompted.

To see this message again, or see help page on any menu, when prompted, type - "help"
To quit, when prompted, type "exit" or "quit"

-----

You Are Currently In -> Main Menu
What Operation Do You Want To Perform? : |
```

Creating New TaskList

```
PowerShell x + v
You Are Currently In -> Main Menu
What Operation Do You Want To Perform? : create
Enter The Name For Task List : Vacation Checklist

-----
New Task List Created!
-----

Press Any Key To Continue
|
```

Viewing Help In Tasklist

```
PowerShell x + v
You Are Currently In -> other chores
What Operation Do You Want To Perform? : help

-----
TaskIt
-----

You are in a Task List.

To create a task, when prompted, type - "create" or "create;<name>" without <>
    When asked about checked, type 'y' for yes or 'n' for no
    When asked about priority, type 0 (None) or 1 (Low) or 2 (Medium) or 3 (High) or 4 (Maximum)
    When asked about tags, type the tags followed by a single ',' (no space after ',')
To see all the tasks, when prompted, type - "view"
To update a task, when prompted, type - "update;<index>" without <>
To delete a task, when prompted, type - "delete" or "delete;<index>" or without <>
To check or uncheck a task, when prompted, type - "check;<index>" or "uncheck;<index>" without <>

To save the database into a '.db' file, type "save" when prompted.
To go back to the main menu, type "home" when prompted.

To see this message again, or see help page on any menu, when prompted, type - "help"
To quit, when prompted, type "exit" or "quit"

-----

Press Any Key To Continue
```

Creating New Task In Tasklist

```
PowerShell x + v
You Are Currently In -> work task list
What Operation Do You Want To Perform? : create
Enter The Name For Task : Make Sequence Flowchart
Checked ? : n
Priority ? : 4
Tags ? : flowchart,work

-----
Task Has Been Created!
-----

Press Any Key To Continue
|
```

Viewing Tasks In Tasklist

PowerShell

×

+

▼

You Are Currently In -> work task list
What Operation Do You Want To Perform? : view

UID	Task Name	Checked	Creation Date	Priority	Tags
0	create project	No	2026-09-19 21:07:14.960831	HIGH	['project', 'work']
1	help friend	Yes	2026-09-19 21:07:39.027900	LOW	['help', 'work']

Press Any Key To Continue

Viewing Tasklists

PowerShell

×

+

▼

You Are Currently In -> Main Menu
What Operation Do You Want To Perform? : view

UID	Task List Name	Total Tasks	Creation Date
0	work task list	3	2026-09-19 21:06:24.785569
1	college tasks	1	2026-09-19 21:06:31.537428
2	other chores	0	2026-09-19 21:06:40.765290
3	Vacation Checklist	0	2026-09-19 21:10:05.917898

Press Any Key To Continue

Saving Data In SQLite3 Database

```
PowerShell
You Are Currently In -> other chores
What Operation Do You Want To Perform? : save

-----
Data Saved Successfully!
-----

Press Any Key To Continue
|
```

Deleting A Tasklist

```
PowerShell
You Are Currently In -> Main Menu
What Operation Do You Want To Perform? : delete;other chores

-----
Task List Has Been Deleted!
-----

-----
Switching To Main Menu
-----

Press Any Key To Continue
|
```

Exiting The Program

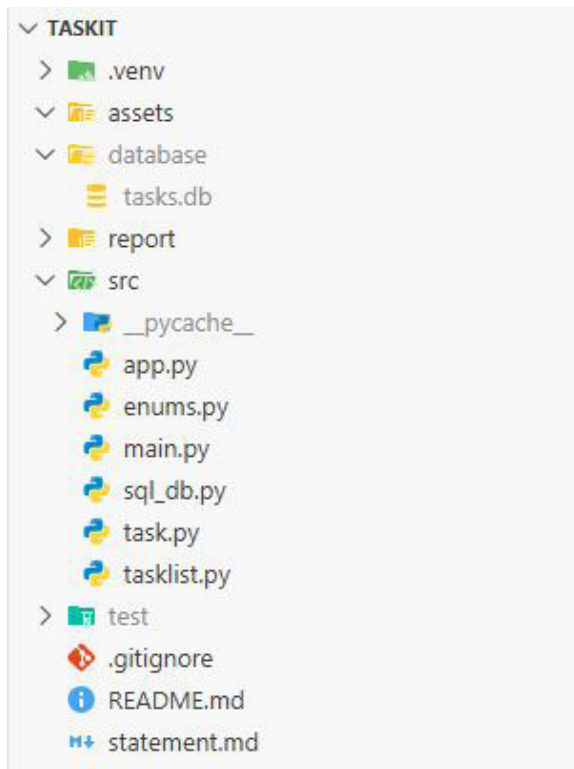
```
PowerShell
You Are Currently In -> Main Menu
What Operation Do You Want To Perform? : exit
Do you want to save before exiting? (Y/N) : y

-----
Data Saved Successfully!
-----

-----
Thank You For Using TaskIt.
-----

PS D:\Code\python\taskit> |
```

Folder Structure



This is the entire code structure as taken from a screenshot from **Visual Studio Code**.

Everything is neatly organized, and everything is made with a purpose in mind.

Testing Approach

- How Was The Code Tested
- How Did It Perform

Testing Approach

Code was mainly tested manually for each and every possible use-case, including SQL injection, String tests, fail-safes, etc.

For example, strings were made to always have an input, i.e. a user cannot enter an invalid / empty string without the system replying appropriately.

Similarly, SQL injection has been prevented by using appropriate method of inputting variables and data into the table.

Test TaskLists and Tasks were made after each change in the program to check whether the program works as intended or not.

No external tool like AI was used in the checking or verification of code at all. Similarly, no testing libraries were used.

The program performed just as well as expected. The results were satisfactory and up to the quality standard that I have set for myself. The results were also up to the mark in terms of their accuracy and speed.

No major issues were found during testing of the program.

The Future

- Challenges Faced
- Learning And Key Takeaways
- Future Enhancements

Challenges Faced

The initial stages of the project had a lot of challenges.

Mainly identifying the proper data storage for the program so it would fit the criteria of being fast, robust, and reliable.

For example, the program initially was meant to have a proper GUI using `PuQt6` but it was rolled back due to time constraints with the project.

Another challenge was the testing and verification of the program's internal workings. During this time, a lot of errors occurred which had to be properly debugged.

One of the biggest challenges was a singular bug that “bugged” me for an entire night. I like to call it the “`Mutable Default Parameter`” bug.

Essentially, assigning a mutable object to a parameter, like an empty list as a default parameter, causes every other instance of that object to hold the same value that was put in the list as it is now linked due to the property of how lists are essentially memory address references.

An easy fix for it was just replacing the mutable object with a `None` object and checking every time if the object was none then replacing it with an empty list later on.

The only other big challenge was of proper parsing of arguments from the user at each step of the loop, which was tackled by using robust checking of different kind of parameters.

Learning And Key Takeaways

In general, a lot of things were new to me and it was really cool to work with them for a semi-big, semi-professional project.

Working more with `Classes` and `Objects` and `Inheritance` taught me a lot of things on how to make a program more modular and easy to use.

Working with `Enum` module taught me how to work with `Enumerators` in `Python`, which made it easier to work as if it was a `struct` from `C` or `C++`.

Creating custom functions like `ArgParse` taught me how to parse a user's argument and how to manipulate and map inputs into useful data. It also taught me how a user might enter data into an input.

Working with `sqlite3` taught me how to work with Relational Databases and implement them directly with `Python` and into the program. This also taught me the importance of selecting an appropriate data structure beforehand during the planning stage of a project.

Working to make a purely `CLI-based-tool` also taught me how the `Command Line` works on the surface level and how different operating systems interpret different commands.

One key takeaway from this is -

`Proper planning of project before the start of it.`

That stage of the project is really important.

Future Enhancements

There are a lot of stages where this program can be enhanced. Some of them I list below -

- A proper GUI, built using existing UI Library like `PyQt6` or `DearPyGui` or similar.
- Introduction of `deadlines` and time gating tasks.
- In relation to the previous point, `OS-Native Notifications` to inform user about a particular task's deadline.
- Exporting of data into formats like `JSON` or `CSV`.
- Possibly a whole rewrite of the program into a more robust and more system-integrated language like `C` or `C++`.

The End

- References
- About Material

References

Python Documentation

<https://docs.python.org/3/>

Python `Classes` Documentation

<https://docs.python.org/3/tutorial/classes.html>

GeeksForGeeks Python

<https://www.geeksforgeeks.org/python/python-programming-language-tutorial/>

W3Schools Python

<https://www.w3schools.com/python/>

SQLite3 Documentation

<https://docs.python.org/3/library/sqlite3.html>

`enum` Documentation

<https://docs.python.org/3/library/enum.html>

`subprocess` Documentation

<https://docs.python.org/3/library/subprocess.html>

`platform` Documentation

<https://docs.python.org/3/library/platform.html>

About

This project was made by me (`real-xp` | `xp`).

My University ID, if required, is -

`26BCE10996`



I did not use my real name in this project as I am not comfortable with sharing my real name on my public `GitHub` account. If you do require my details, mail me.

The repository link for the project is -

`https://github.com/real-xp/taskit`

And my own personal `GitHub` profile is -

`https://github.com/real-xp/`

As said before in Introduction, this project uses no AI in any aspect of it.

Every aspect of the code that was not previously understood, was learned and understood by reading documentation, examples, and watching / reading tutorials and other problems.

The project was entirely made in `Visual Studio Code` without usage of `Intellisense` Or `Auto Code Completion`.

The flowgraphs for the entire PDF were made using `Adobe Illustrator` for Illustration, `yEd Live And Desktop` for Graphs were also used. `Helvetica`, `Carbon Bold` and `FindReplace Black` were the fonts used throughout the document.

In terms of this project PDF, this was made in `Adobe InDesign` as I am more familiar with the program than other PDF making programs.

taskIt

This is a project report on a simple app created by me about allowing easy task management in day to day life.

Everything made in the book is manually typed.

No AI was used in any stage of creation of this book, nor during the creation of the project.

This page is just a simple back-cover page for the entire book / project report. It does not contain anything useful.